



Instituto Politécnico
de Viana do Castelo

Programação I

Ficheiros Binários

Agenda



- Abertura de ficheiros binários: modos de abertura; função de abertura;
- Manipulação de ficheiros binários: funções de leitura e escrita;
- Acesso sequencial e direto;
- Fechar ficheiros;
- Outras funções sobre ficheiros.

Ficheiros Binários



Um ficheiro binário pode ser constituído por qualquer caracter existente na tabela ASCII, podendo conter caracteres especiais. Os ficheiros binários não estão organizados por linhas, são escritos em blocos de memória para o disco e lidos em blocos do disco para a memória.



Modos de abertura de um ficheiro binários são apresentados na tabela seguinte:

Modo	Significado
“rb”	Abre um ficheiro binário em modo de leitura (read). Devolve NULL se não conseguir abrir o ficheiro.
“wb”	Abre um ficheiro binário em modo de escrita (write). É criado um ficheiro com o nome passado na função. Se já existir o ficheiro é truncado. Se não conseguir abrir devolve NULL.
“ab”	Abre um ficheiro binário em modo de acrescentar (append). Se o ficheiro não existir cria-o, se já existir, coloca-se no final de forma a permitir a escrita dos dados a partir dos já existentes.
r+b	Abre ficheiro binário para leitura (read) e escrita (write) (posição=início).
w+b	Cria um ficheiro binário para leitura (read) e escrita (write) (limpa o ficheiro)
a+b	Abre um ficheiro binário para leitura (read) e escrita (write), em modo de “append” (posição =fim).



Nos ficheiros binários a noção de linha não existe, uma vez que os dados não estão organizados por linha.

Nestes ficheiros os dados são escritos em blocos de memória para o disco e lidos em blocos do disco para a memória.

Para escrever blocos de dados em ficheiros está definida a função `fwrite()` que têm a seguinte sintaxe:

```
int fwrite(void *ptr, int size, int num, FILE*fp);
```

Para ler blocos de dados em ficheiros binários está definida a função `fread()`, com a seguinte sintaxe:

```
int fread(void *ptr, int size, int num, FILE *fp);
```



Função `int fwrite(void *ptr, int size, int num, FILE *fp);`

A função recebe quatro parâmetros:

- 1 - um apontador para a informação a escrever no ficheiro (`void *ptr`).
- 2 - o tamanho, em número de bytes, de cada elemento a escrever no ficheiro (`int size`).
- 3 - o número de elementos a escrever no ficheiro (`int num`).
- 4 - o apontador associado ao ficheiro a que se quer aceder (`FILE* fp`).

A função retorna o numero de itens que conseguiu escrever com sucesso.

Exemplo de fwrite: Este programa grava 30 elementos do array Notas, com início no elemento armazenado no índice 10, para o ficheiro binário.

```
#include <stdio.h>
int main() {
    double Notas[100]={23.3,4.2,45.6,45.6,2.3};
    FILE *fp=NULL;
    int result=0;
    // Cria um ficheiro binário para escrita
    fp = fopen("ficheiroTeste.dat", "wb");
    if (fp == NULL) {
        printf("Problemas na CRIACAO do ficheiro\n");
        return -1;
    }
    //Grava 5 números do vetor a partir do índice 0
    result = fwrite (&Notas, sizeof(double), 5, fp);
    printf("Nro de elementos gravados: %d\n", result);
    //Grava 95 números do vetor a partir do índice 6
    result = fwrite (&Notas[5], sizeof(double), 95, fp);
    printf("Nro de elementos gravados: %d\n", result);
    fclose(fp);
    return 0;
}
```

Ficheiros Binários



Para manipular um ficheiro binário considere a utilização de uma aplicação de dump hexadecimal:

Linux:

```
hexdump -Cv ficheiroTeste.dat
```

Windows:

```
certutil -f -encodehex ficheiroTeste.dat
```

O resultado será algo parecido com o que segue:

00000000	cd cc cc cc cc 4c 37 40	cd cc cc cc cc cc 10 40	L7@.....@
00000010	cd cc cc cc cc cc 46 40	cd cc cc cc cc cc 46 40	F@.....F@
00000020	66 66 66 66 66 66 02 40	00 00 00 00 00 00 00 00	fffffff.
00000030	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000040	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000050	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000060	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000070	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000080	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000090	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	



Função `int fread(void *ptr, int size, int num, FILE *fp) :`

A função recebe quatro parâmetros:

- 1 - um apontador para a região de memória que receberá os dados lidos do ficheiro (`void *ptr`).
- 2 - o tamanho, em número de bytes, de cada elemento a ler do ficheiro (`int size`).
- 3 - o número de elementos a ler do ficheiro (`int num`).
- 4 - o apontador associado ao ficheiro a que se quer aceder (`FILE *fp`).

A função retorna o numero de itens que conseguiu ler com sucesso.

Nota: retorna o nº de itens e não o numero de bytes!

Exemplo de fread:
Este programa lê, de
um ficheiro binário,
30 elementos do tipo
double, armazena-os
em memória no array
val e em seguida
escreve-os no ecrã.

```
#include <stdio.h>
int main() {
    FILE *fp=NULL;
    double val[100];
    int result=0,i=0;
    // Abrir ficheiro BINÁRIO para LEITURA
    fp = fopen("ficheiroTeste.dat", "rb");
    if (fp == NULL) {
        printf("Problemas na abertura do ficheiro\n");
        return -1;
    }
    result = fread (&val[0], sizeof(double), 100, fp);
    for (i=0; i<result; i++){
        printf("%f\n", val[i]);
    }
    printf("Nro de elementos lidos: %d\n", result);
    fclose(fp);
}
```



Até aqui, o acesso a ficheiros tem sido feito de forma sequencial, isto é, acede-se a um elemento (leitura ou escrita) e o apontador para o ficheiro fica a apontar para o elemento imediatamente a seguir.

Quando se quer aceder aos elementos de um ficheiro de forma não sequencial temos de reposicionar o apontador de ficheiro para a posição pretendida usando a função `fseek()`, que está declarada da seguinte forma:

```
int fseek(FILE *fp, long num_bytes, int ponto_partida);
```

`fp` é o apontador para o ficheiro devolvido pela função `fopen()`.

A função `fseek()` coloca o apontador de ficheiro na posição `num_bytes` a partir da origem. A origem, ou ponto de partida, pode ter o valor 0, 1 ou 2 consoante a posição pretendida, estando definidas constantes para cada um dos valores.



Os valores possíveis para origem são:

0 - `SEEK_SET` - o salto em `num_bytes`, é realizado a partir da origem do ficheiro

1 - `SEEK_CUR` - o salto em `num_bytes`, é realizado a partir da posição corrente do ficheiro

2 - `SEEK_END` - o salto em `num_bytes`, é realizado a partir do final do ficheiro

Esta função `fseek()` devolve 0 se o movimento dentro do ficheiro foi realizado com sucesso, caso contrário devolve um valor diferente de 0.

O uso de `fseek()` não é muito utilizado com ficheiros de texto sendo mais adequado para ficheiros binários.

Exemplo uso
fseek(): Para ler e
escrever para o
ecrã o 34º byte de
um ficheiro binário.

NOTA: O uso do
modificador L à
constante 34 para
forçar o compilador
a tratar o valor
como um long.

```
#include <stdio.h>
int main(){
    FILE *fp=NULL;
    int ch;
    if((fp=fopen("ficheiroTeste.dat","rb")) == NULL){
        printf("\nErro ao abrir ficheiro!\n");
    }
    else{
        fseek(fp, 34L, 0);
        ch=getc(fp);
        printf("%d",ch);
        fclose(fp);
    }
}
```



Para saber qual a posição que ocupa atualmente num ficheiro posso recorrer à função `ftell` cuja sintaxe é:

```
long ftell(FILE *fich)
```

Esta função devolve um `long` pois a dimensão do ficheiro pode ultrapassar o valor máximo representado por um inteiro.

Independentemente do local onde nos encontramos, é sempre possível voltar a colocar o apontador a apontar para o início do ficheiro através da função `rewind()`, evitando ter de fechar o ficheiro e voltar a abrir para simplesmente colocar o apontador no início do ficheiro. A sua sintaxe é:

```
void rewind(FILE *fich)
```

Ficheiros Binários

Programa que abre
um ficheiro binário e
indica quantos
bytes contem,
obtendo este valor
através de acesso
sequencial.

```
#include <stdio.h>
int main( ){
    FILE *fp=NULL;
    long nBytes=0;
    if ((fp = fopen("ficheiroTeste.dat","rb"))==NULL) {
        printf("Impossível Abrir o ficheiro!\n");
        return -1;
    }
    /* Acesso Sequencial Byte a Byte */
    while (fgetc(fp)!=EOF){
        /* Ler um char */
        nBytes++;
    }
    fclose(fp);
    printf("Dimensao do Ficheiro = %ld\n",nBytes);
    return 0;
}
```



Exemplo de um programa que abre um ficheiro, cujo nome é recebido na linha de comando e indica quantos bytes este contém por meio do acesso direto, isto é, sem ter que percorrer todo o ficheiro.

```
#include <stdio.h>
int main(int argc, char*argv[]) {
    FILE *fp=NULL;
    if (argc!=2) {
        printf("Sintaxe:\n%s <nome ficheiro>\n",argv[0]);
        return -1;
    }
    if ((fp = fopen(argv[1],"rb"))==NULL) {
        printf("Impossível Abrir o ficheiro %s\n",argv[1]);
        return -2;
    }
    /* Ir para o fim do ficheiro */
    fseek(fp,0L,SEEK_END);
    printf("Dimensão do Ficheiro: %ld\n",ftell(fp));
    fclose(fp);
    return 0;
}
```



Outras funções sobre ficheiros:

`int fflush(FILE *fich)`, permite ao programador gravar fisicamente os dados que existem em memória mantendo o ficheiro aberto.

`int fcloseall()`, permite fechar todos os ficheiros abertos.

`int flushall()`, permite efetuar o flush a todos os ficheiros.

`int remove(char *fileName)`, permite apagar do sistema o ficheiro "filename".

`int rename(char *oldFileName, char *newFileName)`, permite mudar o nome do ficheiro "oldFileName" para "newFileName".

Ficheiros Binários

Exemplo: No programa ilustra-se a escrita e leitura de estruturas em ficheiros.

```
#include <stdio.h>
#include <string.h>
typedef struct pes {
    char nome[80];
    float alt;
}PESSOA;

void main() {
    FILE *fp=NULL;
    PESSOA p,r;
    fp = fopen("exemplo.dat","wb");
    strcpy(p.nome,"jose manuel");
    p.alt=1.70;
    fwrite(&p,sizeof(PESSOA),1,fp);
    fclose(fp);
    fp = fopen("exemplo.dat","rb");
    fread(&r,sizeof(PESSOA),1,fp);
    fclose(fp);
    printf("\nNome=%s\nAltura=%f\n",r.nome,r.alt);
}
```

Exemplo: No programa
ilustra-se a exibição de um
ficheiro binário de tamanho
indeterminado.

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    FILE *fp=NULL;
    unsigned char *buffer;
    long tamanho;
    if (argc!=2) {
        fprintf(stderr, "Erro nos argumentos !...\n\n");
        return -1;
        /* Termina o Programa */
    }
    fp = fopen(argv[1],"rb");
    if (fp==NULL) {
        fprintf(stderr, "Erro ao abrir o ficheiro %s\n\n",argv[1]);
        return -1;
    }
    fseek(fp, 0L, SEEK_END);
    tamanho = ftell(fp);
    fseek(fp, 0L, SEEK_SET);
    buffer = (unsigned char*) malloc(tamanho);
    fread(buffer, sizeof(unsigned char), tamanho, fp);
    for(int i = 0; i < tamanho; i++) {
        printf("%c", buffer[i]);
    }
    free(buffer);
    fclose(fp);
    return 0;
}
```

o teu • de partida



Instituto Politécnico
de Viana do Castelo

www.ipvc.pt